Mock Exam 1 — Claude Certified Architect

Date: 2026-04-21 **Format:** 4 scenarios × 10 questions = 40 multiple-choice questions **Time:** 60 minutes (exam pressure simulation) **Pass bar:** 32 / 40 (80%) **Rules:**

- Pick exactly one answer per question (A, B, C, or D)
- No penalty for guessing — answer every question
- Scenarios are independent; read each in isolation
- Answer key lives in `mock-exam-1-answers.md` — don't peek until done

Scenario 1 — Customer Support Resolution Agent

You're designing an AI agent for a SaaS company's customer support. Users contact about billing, refunds, account access, and product questions. Company policy:

- Refunds over $500 escalate to a human
- Account deletion requires two-factor re-confirmation
- PII (email, card number, address) must not appear in any log or prompt
- Tools: `get_customer`, `lookup_order`, `process_refund`, `escalate_to_human`

Q1. Which statement best describes the relationship between `tool_choice` and tool scoping?

- **`A)`** Tool scoping controls which tools the agent **sees** at configuration time; `tool_choice` controls which tool it **invokes** on a given request
- **`B)`** They are two names for the same mechanism — both restrict which tools an agent may call during a session
- **`C)`** `tool_choice` is a hard security boundary; scoping is a hint the model may override when it has good reason
- **`D)`** Scoping applies only to MCP-provided tools; `tool_choice` applies only to built-in tools such as Bash or Read

---

Q2. You need to ensure `process_refund` never runs before `get_customer` has completed. Most reliable approach?

- **`A)`** Add a sentence in the system prompt: "Always call `get_customer` before `process_refund`"
- **`B)`** Make `process_refund` programmatically reject requests when no customer record is cached, returning a structured error directing the model to call `get_customer` first
- **`C)`** Use `tool_choice: "any"` to force the model to call tools in a fixed sequential order
- **`D)`** Remove `process_refund` from the scoped tool list until `get_customer` returns, then dynamically re-add it

---

Q3. The payment API returns `{error_type: "rate_limit", retry_after: 30}`. What should the tool do?

- **A)** Return the raw error to the model and let the model decide whether to retry
- **B)** Bubble the error up as a non-retriable structured failure so the session terminates cleanly
- **C)** Raise an exception immediately so the agent session fails with a visible error state
- **D)** Retry inside the tool with backoff up to 3 attempts; surface to the model only on exhaustion as a structured error

---

Q4. A customer says "Refund my last three orders and delete my account." Which design handles this best?

- **A)** Chain the three refunds and the deletion in a single agent turn — the model has all the context it needs
- **B)** Escalate the entire multi-step request immediately — combined requests are inherently ambiguous
- **C)** Process the refunds autonomously; trigger two-factor re-confirmation for the deletion, then invoke `escalate_to_human`
- **D)** Split each refund into a separate per-item confirmation, then handle the deletion last in the same session

---

Q5. Which approach to keeping PII out of logs is INCORRECT?

- **A)** Instruct the model in the system prompt to avoid writing PII into any tool argument or response field
- **B)** Redact at the tool boundary — tools return placeholder tokens to the model; real values stay backend-only
- **C)** Use structured tool outputs that expose only non-PII fields (tier, plan, tenure) to the model
- **D)** Sanitise logs at ingest time by pattern-matching common PII shapes (emails, card numbers) before write

---

Q6. A customer requests a $600 refund on a $700 order. Policy: "refunds over $500 escalate." Best action?

- **A)** Process the $500 autonomously and escalate the remaining $100 to a human
- **B)** Invoke `escalate_to_human` with a structured summary (customer, order, amount, policy trigger) and let the human decide
- **C)** Process the full $600 autonomously with an audit-flag comment noting the policy exception
- **D)** Decline the refund and tell the customer to email support — the policy is unambiguous

---

Q7. `get_customer` currently returns `{id, tier, quota}`. You're considering adding `refund_history_30d` and `open_tickets_count`. Strongest reason?

- **A)** Richer tool output is generally preferable — more context always helps the model make nuanced decisions
- **B)** The fields are required for downstream logging, audit trail, and compliance traceability
- **C)** Both fields meaningfully change the agent's action (refund denial for abusers, escalation for overloaded accounts) — enriched outputs beat multiple tool calls
- **D)** The certification rubric rewards structured outputs with many fields as a signal of good schema design

---

Q8. Handoff summary for human escalation. Which field set best serves the human agent?

- **A)** Full verbatim transcript of the conversation plus every tool call's inputs and outputs
- **B)** Customer name, order ID, and a one-line description — minimalism prevents information overload
- **C)** `{customer_id, category, requested_action, policy_trigger, confidence, suggested_next_step}` plus short narrative and links to the underlying records
- **D)** The model's own prose summary only, with no structured fields — humans prefer reading narrative

---

Q9. When should the support session and the quality-review session that audits agent outputs be kept isolated?

- **A)** Always — session isolation is a general reliability principle regardless of use case
- **B)** Never — sharing session state gives the reviewer full context and improves audit quality
- **C)** Only when the reviewer uses a different system prompt than the support agent
- **D)** When the reviewer shares tools with the support agent and could otherwise trigger duplicate side-effects such as re-escalating an already-escalated ticket

---

Q10. Why include a `detected_pattern` field in structured output alongside `confidence`?

- **A)** `confidence` says how sure the model is; `detected_pattern` says **why** — critical for debugging false positives and triaging the underlying cause
- **B)** `detected_pattern` replaces `confidence`; the two are redundant and `confidence` is deprecated as of the latest API
- **C)** The Claude Agent SDK requires `detected_pattern` whenever `tool_use` is active for audit purposes
- **D)** The field is logging-only metadata; the model itself does not read or act on its contents

---

Scenario 3 — Multi-Agent Research System

Your firm's legal team needs a research agent for case law and regulatory text. Queries range from narrow ("all 2020 Federal Court rulings on GDPR applicability in Germany") to broad ("overview of EU data privacy law"). Subagents available: `case-law-search`, `regulatory-text`, `legal-commentary`, `foreign-law`, `statistical-summary`.

Q11. Primary job of the coordinator agent?

- **A)** Forward the query verbatim to one subagent at a time in a fixed sequence until an answer appears
- **B)** Gather raw text from every subagent and concatenate it for the user without any filtering step
- **C)** Run all five subagents in parallel on every query to maximise coverage and let the user filter noise
- **D)** Analyse the query, select a matching subset of subagents, synthesise their structured findings, and decide when coverage is sufficient

---

Q12. Query: "List all 2020 Federal Court rulings on GDPR in Germany." Best subagent selection?

- **A)** All five subagents — breadth ensures no relevant material is missed on a regulatory question like this
- **B)** `case-law-search`, `regulatory-text`, and `legal-commentary` — case-law questions always need regulatory and commentary framing
- **C)** `regulatory-text` alone — GDPR is primarily a regulation, not a body of case law, so that covers it
- **D)** `case-law-search` and `foreign-law` — scope is narrow (2020, Germany, GDPR); additional subagents add noise

---

Q13. Which subagent-output field set is essential for the coordinator to aggregate without cross-origin confusion?

- **A)** `{source, finding, evidence, confidence}` — self-attribution lets the coordinator cite and resolve conflicts without tracking origin externally
- **B)** `{finding, confidence}` — the coordinator remembers which agent produced which finding via its own conversation state
- **C)** `{agent_id, text}` — the coordinator looks up the agent by ID to get context when needed
- **D)** `{finding}` alone — additional fields dilute the signal and inflate context window size unnecessarily

---

Q14. The coordinator passes a subagent's output to a downstream summariser. When should it pass raw text vs a structured summary?

- **A)** Always raw — summarisation loses information; the downstream agent should decide what to keep
- **B)** Summary by default; pass raw text only when the downstream agent must **reason over the details** — for example a compliance check over full contract language
- **C)** Always summary — raw text bloats context and most downstream consumers don't need source-level fidelity
- **D)** Raw for short outputs under 1k tokens; summary otherwise — the size threshold is the deciding factor

---

Q15. Best stopping condition for the coordinator's iterative re-delegation loop?

- **A)** Fixed max iteration count (e.g. 5) — bounds cost and prevents runaway loops regardless of progress
- **B)** Whichever comes first: 3 iterations or a human approval signal in the loop
- **C)** A quality threshold on synthesis coverage of the original query; exit when coverage is adequate, else re-delegate targeted gaps
- **D)** No explicit stopping condition; the coordinator loops until a subagent reports "no more information"

Q16. Why distribute tool access across subagents instead of pooling the union into each one?

- **A)** To reduce token cost from repeated tool schemas across each subagent's context window
- **B)** To enforce least privilege — a subagent with only the tools it needs cannot drift outside its specialisation, and scoping is a hard security boundary
- **C)** To simplify debugging by constraining which tool each subagent can be held accountable for when things go wrong
- **D)** Tool distribution is a stylistic preference; pooling the full union into each subagent is functionally equivalent

---

Q17. Mid-research the synthesis has a gap on a specific German Federal Court's 2020 rulings. What's the right re-delegation?

- **A)** Send the gap back to all five subagents and take the union of new findings as additional coverage
- **B)** Escalate to human review — the model cannot meaningfully improve on its first attempt at this narrow scope
- **C)** Send a targeted narrower query to `case-law-search` with the specific court and year constraints; skip subagents already delivering adequate coverage
- **D)** Increase `max_tokens` on the coordinator and rerun the entire multi-subagent query from scratch

---

Q18. Two subagents return findings that contradict each other on the same point. Coordinator action?

- **A)** Pick the subagent with the higher stated `confidence` value and discard the other finding
- **B)** Treat both findings as present and note the conflict in synthesis with both sources cited, letting the downstream human adjudicate
- **C)** Re-delegate to a third independent subagent as a tiebreaker — the majority answer wins on merit
- **D)** Discard both findings entirely — contradiction means both are unreliable and shouldn't appear in output

---

Q19. Which statement about tool scoping across subagents is INCORRECT?

- **A)** A subagent restricted to read-only tools cannot be prompt-injected into performing writes on the system
- **B)** Scoping is enforced at configuration time, before any prompt from the user ever reaches the model
- **C)** Scoping applies equally to built-in tools (Read, Bash) and MCP-provided tools — same enforcement path
- **D)** A subagent with scoping can still call a tool outside its scope if the user's request explicitly asks for it

---

Q20. Which metric best captures "the research system is working well"?

- **A)** Coverage of the original query in the synthesis, cited against sources, with explicit uncovered-gap notes
- **B)** Total number of subagent calls per user query — higher implies more thorough research on the topic
- **C)** End-to-end wall-clock time per user query — speed matters in legal workflows with tight deadlines
- **D)** Token cost per user query — lower is better regardless of quality since the downstream review is what matters

---

Scenario 4 — Developer Productivity with Agent SDK

You're building a code-review agent using the Claude Agent SDK. It reads a repo, runs tests, posts comments to GitHub, and learns from past reviewer feedback. Built-in tools: Read, Write, Edit, Bash, Grep, Glob.

Q21. You need to find every file in the repo that contains the string `"deprecated"`. Best tool and configuration?

- **`A)`** Grep with pattern `"deprecated"` and `output_mode: "files_with_matches"` — content search with a path-only result
- **`B)`** Glob with pattern `**/deprecated` — matches files whose **names** contain "deprecated" across the tree
- **`C)`** Bash with `find . -name 'deprecated'` — finds files by name pattern recursively through the directory
- **`D)`** Read on each file in a loop, inspecting contents for the string — explicit iteration over the repo tree

---

Q22. You need to change line 42 of `src/config.py` from `DEBUG = True` to `DEBUG = False`. Best tool?

- **`A)`** Write — overwrite the whole file with the corrected contents and trust the model to preserve everything else
- **`B)`** Bash with `sed -i 's/DEBUG = True/DEBUG = False/' src/config.py` — external substitution is faster than tool calls
- **`C)`** Read the file, mutate the string in memory, and Write it back — keeps the transformation in the agent's context
- **`D)`** Edit — express the change as a find-and-replace on the exact line, minimising diff noise and accidental edits

---

Q23. You need to run `pytest tests/` and capture the result for the agent to inspect. Best mechanism?

- **`A)`** MCP server named `python-tests` wrapping pytest invocation — avoid Bash for language-specific tooling
- **`B)`** Read `tests/` via Glob, then simulate the test run by inspecting assertions and imports inline
- **`C)`** Write a shell script to `/tmp/run.sh` and exec it via Bash — wrapping improves reproducibility across runs
- **`D)`** Bash tool with `command: "pytest tests/"` and capture stdout via the tool result

---

Q24. The agent needs to post comments to GitHub. Best MCP-server access level?

- **A)** Full GitHub MCP with all repo read, write, and admin scopes enabled — flexibility matters more than restriction
- **B)** Scoped GitHub MCP with only `issues:write`, `pull_requests:write`, `contents:read` — least privilege matches the agent's actual needs
- **C)** A shell wrapper over the `gh` CLI via Bash — simpler than configuring MCP scopes and more flexible per-call
- **D)** A custom MCP server written per-repo, hardcoding the specific repository ID inside each build

---

Q25. The same agent that generates code also reviews it before submission. Why is this design problematic?

- **A)** It isn't problematic — running generation and review in a single session is more token-efficient overall
- **B)** The two sessions have incompatible tool scopes, which causes frequent tool-call errors at runtime
- **C)** The reviewing session shares context with the generating session, anchors on the same reasoning path, and misses issues the original missed — session isolation restores reviewer independence
- **D)** The Claude Agent SDK forbids multi-step generate-then-review workflows in a single active session

---

Q26. Designing review severity categories for the agent's output. Best approach?

- **A)** Enum {critical, warning, info} with few-shot examples anchoring what each severity means in context — prevents calibration drift and downstream parsing errors
- **B)** Free-text `severity` field — let the model generate whatever label best fits the individual finding
- **C)** Numeric 0-10 scale with no anchoring examples — numerical precision communicates gradation cleanly
- **D)** Binary pass/fail categorisation — keeps the output simple and avoids category confusion entirely

---

Q27. CI pipeline invokes Claude Code to review a PR. The output must be machine-readable. Which flag set?

- **A)** `--interactive=false --log-level=none` — strips interactive prompts and noisy logging
- **B)** `--output-format markdown --human-readable` — markdown is trivially machine-parseable with a library
- **C)** `-p <prompt>` alone — stdout is parseable by default for any downstream consumer
- **D)** `-p <prompt> --output-format json --json-schema <schema>` — structured output against a declared schema

Q28. Your code-review agent produces output in a fixed format via `tool_use`. Which statement is INCORRECT?

- **A)** Using `tool_use` for the output gives API-level schema enforcement stronger than system-prompt instructions alone
- **B)** Once `tool_choice` forces the emit tool, validation of the tool's argument values becomes unnecessary downstream
- **C)** `tool_choice` prevents hallucinations about **which** tool to call, but cannot prevent field-level hallucination **inside** the tool's arguments
- **D)** Forcing `tool_choice: {type: "tool", name: "emit_review"}` guarantees the model calls the emit tool before responding

---

Q29. Grep found matches across the repo. You need only file paths, not surrounding content. Best `output_mode`?

- **A)** `content` — it's the default, any filtering happens client-side after the result returns from the tool
- **B)** `count` — counts matches per file first, then you derive the file list from the non-zero counts
- **C)** `files_with_matches` — returns file paths only, saves tokens, and supports `head_limit` for truncation
- **D)** `filenames_only` — explicit mode designed for this common file-listing case

---

Q30. A single code-review agent runs in CI across 50 repos, each with different review criteria. Best MCP scoping approach?

- **A)** Per-repo scoped MCP configuration — each invocation instantiates tools parameterised for that repo, nothing else
- **B)** One global MCP server with all criteria baked in; the agent selects the matching subset per repo via prompting
- **C)** No MCP at all — put all review criteria into CLAUDE.md and rely on the model to follow the repo-specific sections
- **D)** Single agent with no scoping — let the model figure out which tools are applicable based on repository context

---

Scenario 6 — Structured Data Extraction

You're extracting contract terms from signed PDFs: party names, effective date, payment terms (enum: monthly/quarterly/annually), payment amount, currency, renewal clause (enum: auto-renew/manual-renew/expire). PDFs vary in structure. A downstream accounting system requires strict JSON conforming to a published schema.

Q31. What gives you the strongest guarantee of schema compliance?

- **A)** Detailed system-prompt instructions specifying exact field names, types, and the JSON structure to emit
- **B)** Define the schema as a Claude `tool_use` input schema and force `tool_choice` on that tool — API-level JSON Schema enforcement, not a prompt convention
- **C)** Few-shot examples in the system prompt showing the valid JSON shape the model should produce
- **D)** Validate the output post-hoc and retry on schema mismatch; no upfront enforcement is strictly needed

---

Q32. A contract specifies payment frequency as "bi-monthly" (not in the enum). Best schema design?

- **A)** Widen the enum to include every plausible payment frequency — enumerate all permutations encountered
- **B)** Keep the enum as is — the model will pick the closest valid value automatically, which will be close enough
- **C)** Add an "other" option to the enum plus an optional `payment_frequency_detail` free-text field for capture when "other" is selected — prevents the enum-hallucination trap
- **D)** Omit the enum entirely; let the model return free-text and map the value to the enumeration downstream

---

Q33. How to decide whether a field should be required vs optional in the schema?

- **A)** Required if the downstream system cannot function without the field; optional otherwise — downstream-blocking is the test
- **B)** All fields should be required to keep the schema strict and catch incomplete extractions upfront
- **C)** Optional by default; required only when the field appears in every document type observed to date
- **D)** Required for primitive types (string, number); optional for nested objects and arrays

---

Q34. A contract says "effective date: 2023-01-15" but the signature date on page one is "2023-02-01". Business rule: effective date cannot precede signature date. Where to handle?

- **A)** Embed the business rule into the JSON Schema via a custom validator — keeps extraction and validation in one atomic pass
- **B)** Separate passes: extraction records both dates verbatim; a distinct validation pass flags the contradiction without forcing the model to silently invent a resolution
- **C)** Ignore the contradiction at extraction time — extract the literal value and let downstream catch it
- **D)** Add a system-prompt instruction: "If dates contradict, always use the later of the two as the effective date"

Q35. Dates come in varied formats (`Jan 5, 2023`, `05/01/2023`, `2023-01-05`). Where should normalisation live?

- **A)** Ask the model to pick a format on each call — flexibility prevents false positives from rigid rules
- **B)** In the downstream accounting system — the extraction agent should return whatever the source document literally says
- **C)** In a post-extraction rule engine that runs after the agent has emitted the raw string value
- **D)** In each parameter description of the tool schema, specifying the target format (ISO 8601) with examples of the transformation

---

Q36. First extraction errors: a date string is malformed. Should the agent retry?

- **A)** Retry always — the model can self-correct on a second pass given the error message appended to the prompt
- **B)** Never retry — if the first extraction fails, escalate to human review immediately to avoid compounding errors
- **C)** Triage first: if the error is a model parse mistake on data that exists in the source, retry with the error appended; if the source actually has no parseable date, do not retry — no retry invents data
- **D)** Retry up to 5 times with exponential backoff regardless of error type — standard transient-error handling

---

Q37. Few-shot examples of contract-extraction edge cases. Best placement?

- **A)** System prompt or tool description — the examples anchor the model's calibration across all calls and travel with the agent definition, not the individual request
- **B)** Per-request, inserted just above the current document being processed — matches the task's exact content each time
- **C)** In a separate MCP tool that the agent calls when it encounters ambiguity during extraction
- **D)** Neither — few-shot examples bias the output; a sufficiently clear schema is enough on its own

---

Q38. `tool_choice` configuration for this extraction task?

- **A)** `auto` — let the model decide whether to call the extraction tool or respond directly with prose
- **B)** `any` — force a tool call but let the model pick which tool to invoke on each document
- **C)** `{type: "tool", name: "extract_contract"}` — force the specific extraction tool on every call; the downstream system has no use for prose responses
- **D)** `none` — extraction should be free-form text parsed by a dedicated downstream parser

---

Q39. Why include a `detected_pattern` field in the extraction output schema?

- **A)** It is required by the Anthropic API for all structured extraction tasks using `tool_use`
- **B)** It exposes which textual cue in the source document triggered a given classification — debuggable false positives, targeted prompt fixes instead of wholesale reruns
- **C)** It substitutes for a confidence score and is more machine-parseable than a floating-point value
- **D)** It is a legacy field from older Claude schema versions and should be omitted in new designs

---

Q40. "Effective date" is downstream-blocking — without it accounting cannot post the contract. Schema choice?

- **A)** Optional with a default of today's date — gives the downstream system something workable to proceed with
- **B)** Required with a fallback of the signature date — graceful degradation for documents with unclear effective dates
- **C)** Optional with a non-null `detected_pattern` note explaining why extraction failed to find the value
- **D)** Required with no default — absence is an explicit extraction failure the downstream system can detect and route to human review

---

End of exam

Post your A/B/C/D answers in order (1–40). I'll grade, compute per-scenario and per-domain breakdowns, and produce the readiness report for `artefacts/claude-certified-architect/phase-3/readiness-report.md`.